UNIVERSIDAD ANÁHUAC MAYAB
Ingeniería en Tecnologías de la Información y Negocios Digitales

Bitácora

Diario personal con fotos, dictado por voz y geolocalización

Reporte técnico · Parcial 2

Alumno	Marcos
Matrícula	
Profesor	
Tecnología	Flutter / Dart · SQLite
Fecha	Noviembre 2026

1. Justificación del proyecto

1.1 Problema

Llevar un diario personal tiene beneficios conocidos: ayuda a ordenar ideas, a reducir el estrés y a conservar recuerdos que con el tiempo se olvidan. Sin embargo, la mayoría de las personas abandona el hábito por tres razones prácticas:

- **Escribir toma tiempo.** Al final del día, teclear varios párrafos en el teléfono es lento y cansado.
- **El texto solo pierde contexto.** Meses después es difícil recordar *dónde* ocurrió algo o *cómo se veía*.
- **Las alternativas están dispersas.** Las fotos quedan en la galería, la ubicación en otra app y las notas en otra; nada está conectado.

1.2 Solución

Bitácora reúne en una sola entrada todo lo que describe un día: el texto, una foto y el lugar. Para que registrar sea rápido, aprovecha el hardware del teléfono:

Gadget	Cómo resuelve el problema
Micrófono + reconocimiento de voz	El usuario **dicta** su día en lugar de escribirlo; el texto aparece en tiempo real y se puede corregir a mano.
Cámara y galería	Una foto tomada al momento (o elegida de la galería) ilustra la entrada y ayuda a recordarla.
GPS + mapas	Al guardar, la app registra automáticamente la ubicación; en el detalle se ve en un mapa interactivo y se puede abrir en Google Maps.

Toda la información se guarda **localmente en SQLite**: la app funciona sin conexión y los datos personales no salen del dispositivo, algo importante tratándose de un diario.

1.3 Usuarios y casos de uso

- Personas que quieren mantener un diario personal sin invertir mucho tiempo.
- Viajeros que desean una bitácora de viaje con fotos y mapa de cada lugar visitado.
- Estudiantes o profesionales que hacen trabajo de campo (visitas, inspecciones, prácticas) y necesitan notas con evidencia fotográfica y ubicación.

2. Funcionalidades

Pantalla	Descripción	Requerimiento cubierto
Mis entradas (HomeScreen)	Lista de entradas ordenadas de la más reciente a la más antigua, con miniatura, fecha e indicador de ubicación. Permite recargar jalando hacia abajo.	Scaffold, AppBar, Drawer, FutureBuilder
Nueva entrada (NewEntryScreen)	Formulario con validación: foto (cámara o galería), título, contenido escrito o dictado. Al guardar obtiene el GPS y almacena todo en SQLite.	Cámara, galería, micrófono, GPS, try-catch
Detalle (EntryDetailScreen)	Foto en grande con animación Hero, texto completo, fecha, mapa interactivo con marcador y botón “Abrir en Google Maps”. Permite eliminar la entrada.	Hero, mapa, Google Maps
Acerca de (AboutScreen)	Información de la app accesible desde el Drawer.	Drawer, rutas

3. Arquitectura

3.1 Organización por capas

El código se organiza en capas con dependencias en un solo sentido: la presentación depende de abstracciones del dominio, y las implementaciones concretas (SQLite, plugins de hardware) se eligen en un único punto, el `ServiceLocator`.

Capa	Carpeta	Responsabilidad
Dominio	`lib/domain/`	Entidades (`DiaryEntry`, `GeoPoint`) y contrato `DiaryRepository`. No importa Flutter ni ningún plugin.
Datos	`lib/data/`	`DatabaseHelper` abre la BD y crea el esquema; `SqliteDiaryRepository` implementa las consultas.
Servicios	`lib/services/`	Acceso al hardware: cámara/galería, micrófono, GPS y apertura de Google Maps. Cada uno con su interfaz.
Presentación	`lib/presentation/`	Pantallas y widgets reutilizables.
Core	`lib/core/`	Rutas, tema, excepciones, utilidades e inyección de dependencias.

3.2 Principios SOLID aplicados

Principio	Aplicación en el proyecto
S – Responsabilidad única	`DatabaseHelper` sólo gestiona la conexión; `SqliteDiaryRepository` sólo hace consultas; `DateFormatter` sólo formatea fechas; cada servicio controla un solo gadget.
O – Abierto/cerrado	Para cambiar SQLite por otra tecnología se crea una nueva clase que implemente `DiaryRepository`, sin modificar las pantallas.
L – Sustitución de Liskov	Cualquier implementación de `LocationService`, `MediaService`, etc. puede reemplazar a otra (por ejemplo, versiones falsas en pruebas).
I – Segregación de interfaces	Interfaces pequeñas por gadget en lugar de un único “servicio de hardware”. `HomeScreen` sólo depende del repositorio.
D – Inversión de dependencias	Las pantallas reciben abstracciones por constructor. Sólo `ServiceLocator.production()` conoce las clases concretas.

3.3 Diagrama de clases

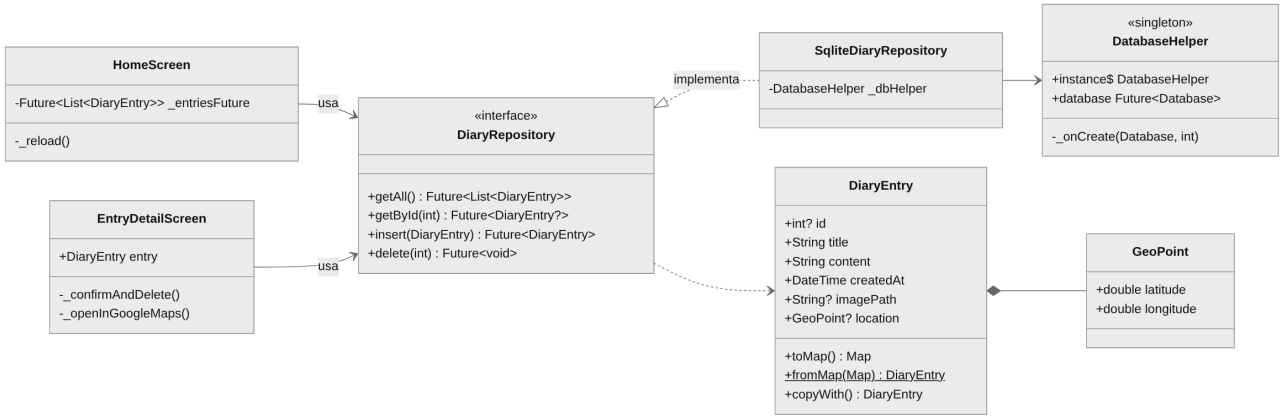


Figura 1. Dominio, persistencia y pantallas que consumen el repositorio.

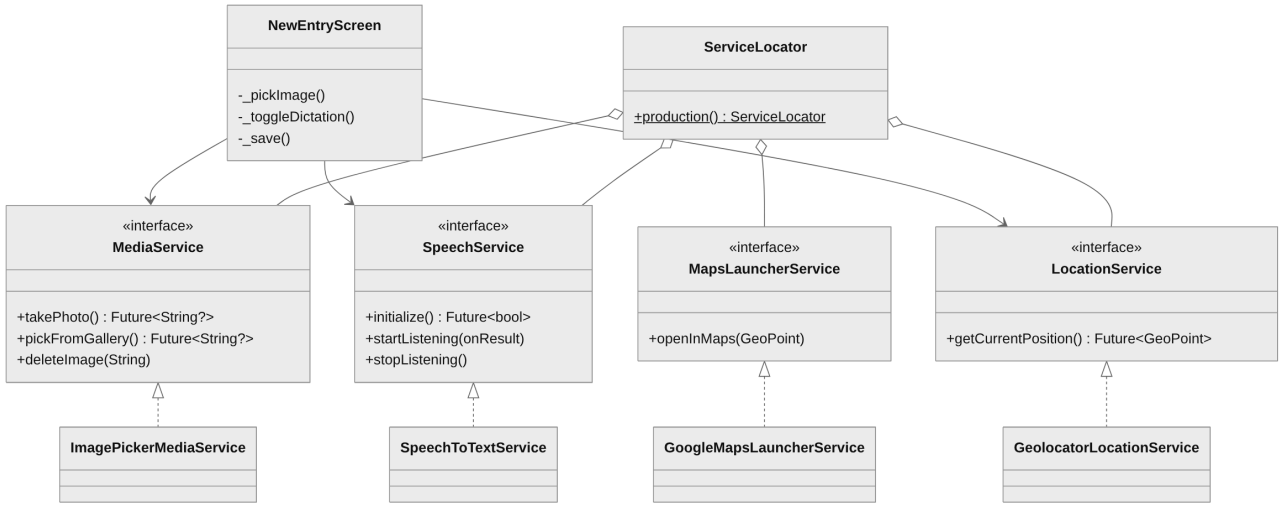


Figura 2. Servicios de hardware (interfaz + implementación) e inyección de dependencias.

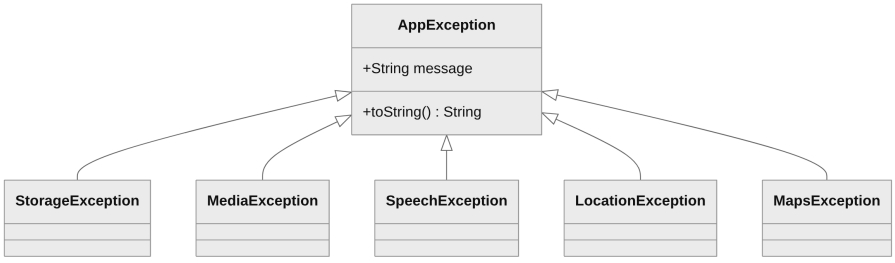


Figura 3. Jerarquía de excepciones de la aplicación.

3.4 Flujo de datos

Los datos viajan del hardware a los servicios, que los entregan ya procesados al formulario (ruta de la imagen, texto reconocido, coordenadas). El formulario construye una `DiaryEntry` y la envía al repositorio, que la inserta en SQLite. La lista consulta el repositorio de forma asíncrona y, al tocar una entrada, la pasa como argumento de la ruta al detalle.

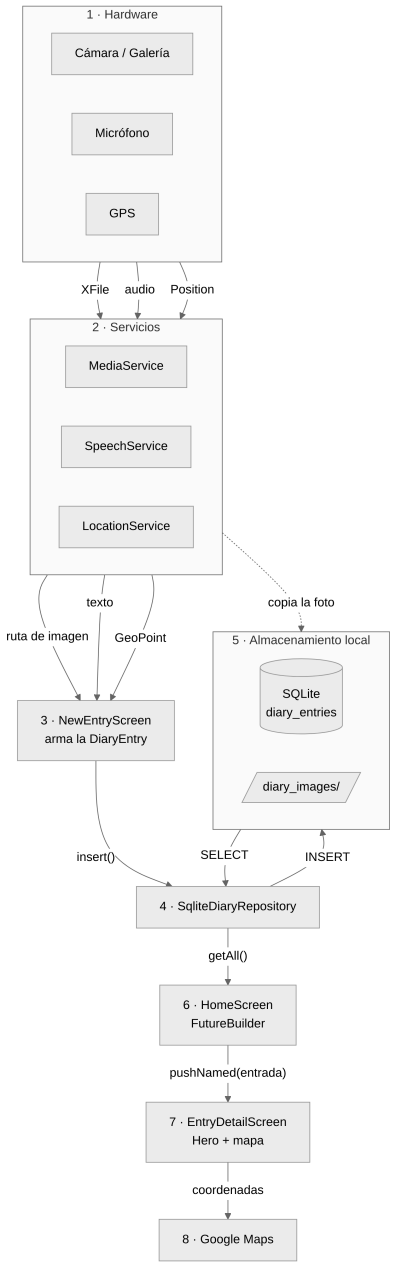


Figura 4. Flujo de datos de la aplicación.

3.5 Secuencia: crear una entrada

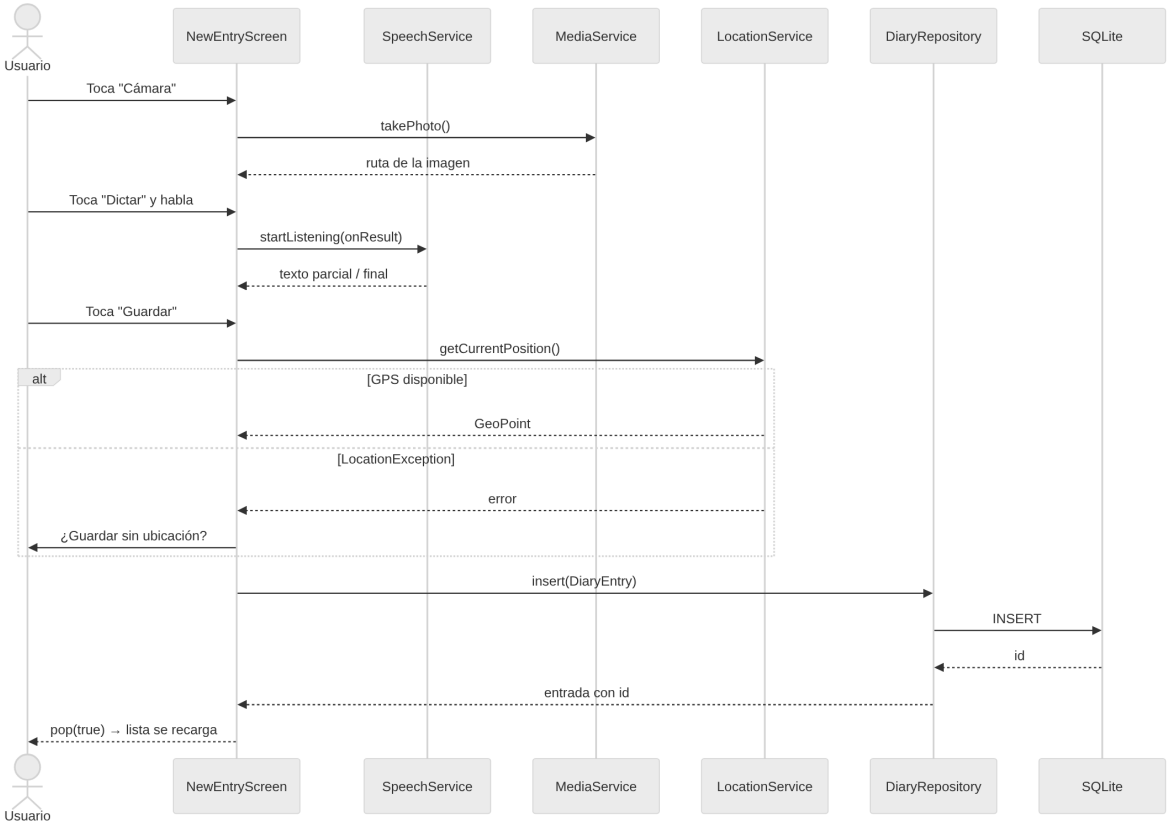


Figura 5. Interacción entre componentes al crear y guardar una entrada.

4. Almacenamiento local

Se eligió **SQLite** (paquete `sqflite`) en lugar de SharedPreferences porque los datos son registros estructurados que se consultan ordenados por fecha; SharedPreferences está pensado para pares clave-valor simples (configuraciones).

```
CREATE TABLE diary_entries (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  title       TEXT    NOT NULL,
  content     TEXT    NOT NULL,
  created_at  INTEGER NOT NULL,   -- milisegundos desde epoch
  image_path  TEXT,               -- ruta en <documentos>/diary_images/
  latitude    REAL,
  longitude   REAL
);
CREATE INDEX idx_entries_created_at ON diary_entries(created_at DESC);
```

Las imágenes no se guardan dentro de la base de datos (sería ineficiente); se **copian** a la carpeta de documentos de la app y en SQLite se guarda sólo su ruta. Esto es necesario porque `image_picker`

entrega archivos temporales que el sistema puede borrar. Al eliminar una entrada también se elimina su foto.

5. Manejo de errores y carga asíncrona

- **Excepciones propias:** `AppException` y sus subclases (`StorageException`, `LocationException`, `MediaException`, `SpeechException`, `MapsException`). Cada capa atrapa los errores técnicos con `try-catch` y los traduce a una excepción con un mensaje en español.
- **Presentación:** las pantallas sólo atrapan `AppException` y muestran el mensaje en un `SnackBar` o diálogo; la app nunca se cierra por un error de hardware.
- **GPS opcional:** si el GPS está apagado o se niega el permiso, se pregunta al usuario si desea guardar la entrada sin ubicación, en lugar de perder lo que escribió.
- **FutureBuilder:** `HomeScreen` muestra tres estados: cargando (indicador de progreso), error (mensaje con botón “Reintentar”) y datos (lista o estado vacío).
- **Contexto seguro:** después de cada `await` se verifica `mounted` / `context.mounted` antes de usar el `BuildContext`.

6. Navegación y animaciones

La navegación usa **rutas con nombre** definidas en `AppRoutes` y un generador (`onGenerateRoute`) que permite pasar argumentos tipados, como la entrada que se abre en el detalle. Las pantallas devuelven un resultado con `Navigator.pop(context, true)` para que la lista se recargue sólo cuando hubo cambios.

Ruta	Pantalla	Transición
`/`	HomeScreen	Material estándar
`/nueva-entrada`	NewEntryScreen	Deslizamiento desde abajo (`SlideTransition`)
`/detalle`	EntryDetailScreen	Desvanecimiento (`FadeTransition`) + **Hero** de la imagen
`/acerca-de`	AboutScreen	Material estándar

Otras animaciones: `AnimatedSwitcher` al cambiar la foto del formulario y `AnimatedContainer` en el botón de dictado, que cambia de color y tamaño mientras el micrófono escucha.

7. Permisos y paquetes

Paquete	Uso	Permiso Android
`image_picker`	Cámara y galería	`CAMERA`
`speech_to_text`	Captura de audio y voz a texto	`RECORD_AUDIO`
`geolocator`	Posición GPS	`ACCESS_FINE_LOCATION`, `ACCESS_COARSE_LOCATION`
`flutter_map` + `latlong2`	Mapa interactivo (OpenStreetMap, sin API key)	`INTERNET`
`url_launcher`	Abrir Google Maps con las coordenadas	— (declaración `<queries>`)
`sqflite`, `path`, `path_provider`	Base de datos y archivos locales	—

Los permisos se solicitan en tiempo de ejecución justo cuando se necesitan (al tocar la cámara, el micrófono o al guardar), no al abrir la app.

8. Calidad del código

- Comentarios de documentación (`///`) en todas las clases públicas explicando su responsabilidad.
- Reglas de estilo con `flutter_lints`.
- Pruebas unitarias del dominio (`flutter test`): serialización de `DiaryEntry` y formato de fechas.
- Historial de Git con commits descriptivos por capa (configuración, dominio, datos, servicios, navegación, pantallas, pruebas y documentación).

9. Conclusión

Bitácora cumple los requerimientos del parcial integrando tres gadgets del dispositivo (cámara, micrófono y GPS) en un flujo natural para el usuario: tomar una foto, dictar lo que pasó y guardar. La separación en capas y el uso de interfaces permiten cambiar la tecnología de almacenamiento o cualquier plugin sin reescribir la interfaz, y el manejo centralizado de excepciones hace que la app responda de forma clara ante permisos denegados, GPS apagado o fallos de la base de datos.